Introduction

- A model is a simplified version of a real system.
- A **model** is constructed by focusing only on a few aspects of the problem and ignoring the rest.
- The model of a problem is called an ***analysis model***. On the other hand, the model of the solution (code) is called the ***design model***.
- The design model is usually obtained by carrying out iterative refinements to the analysis model using a design methodology.

Unified Modelling Language (UML)

- A model can be documented using a modelling language such as **unified modelling language(UML)**.
- UML has been accepted by ISO as a standard for modelling object-oriented systems.
- UML can be used to document object-oriented analysis and design results that have been obtained using any methodology.

UML Diagrams

- UML 1.0 can be used to construct nine different types of diagrams to capture five different views of a system.
- The different UML diagrams provide different perspectives of a software system to be developed and facilitate a comprehensive understanding of the system.
- Each perspective focuses on some specific aspect and ignores the rest.
- Once a system has been modelled from all the required perspectives, the constructed models can be refined to get the actual implementation of the system.
- UML diagrams can capture the following views (models) of a system:
    - User's view
    - Structural view
    - Behaviourial view
    - Implementation view
    - Environmental view

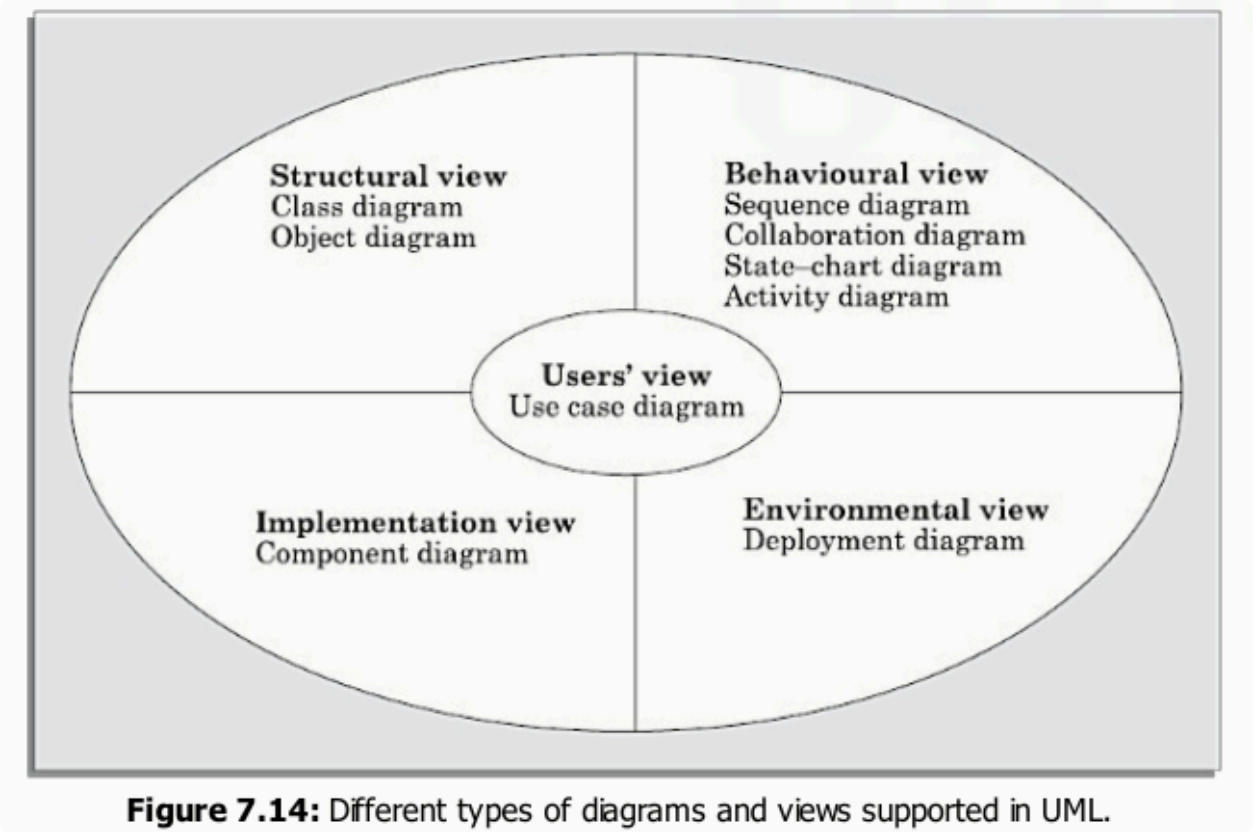


Figure 7.14: Different types of diagrams and views supported in UML.

- Users' view is shown as the central view because based on the users' view, all other views are developed and all views need to conform to the user's view.
- Most of the object oriented analysis and design methodologies require us to iterate among the different views several times to arrive at the final design.

User's view

- This view defines the functionalities made available by the system to its users.
- The users' view is a black-box view of the system where the internal structure, the dynamic behaviour of different system components, the implementation etc. are not captured.
- The users' view is very different from all other views in the sense that it is a **functional model** compared to all other views that are essentially **object models**.

Structural view

- The structural view defines the structure of the problem (or the solution) in terms of the kinds of objects (classes) important to the understanding of the working of a system and to its implementation.
- It also captures the relationships among the classes (objects).

- The structural model is also called the ***static model***, since the structure of a system does not change with time.

Behaviourial view

- The behaviourial view captures how objects interact with each other in time to realise the system behaviour.
- The system behaviour captures the time-dependent (dynamic) behaviour of the system. It therefore constitutes the dynamic model of the system.

Implementation view

- This view captures the important components of the system and their interdependencies.
- For example, the implementation view might show the GUI part, the middleware, and the database part as the different parts and also would capture their interdependencies.

Environmental view

- This view models how the different components are implemented on different pieces of hardware.

Use Case Model

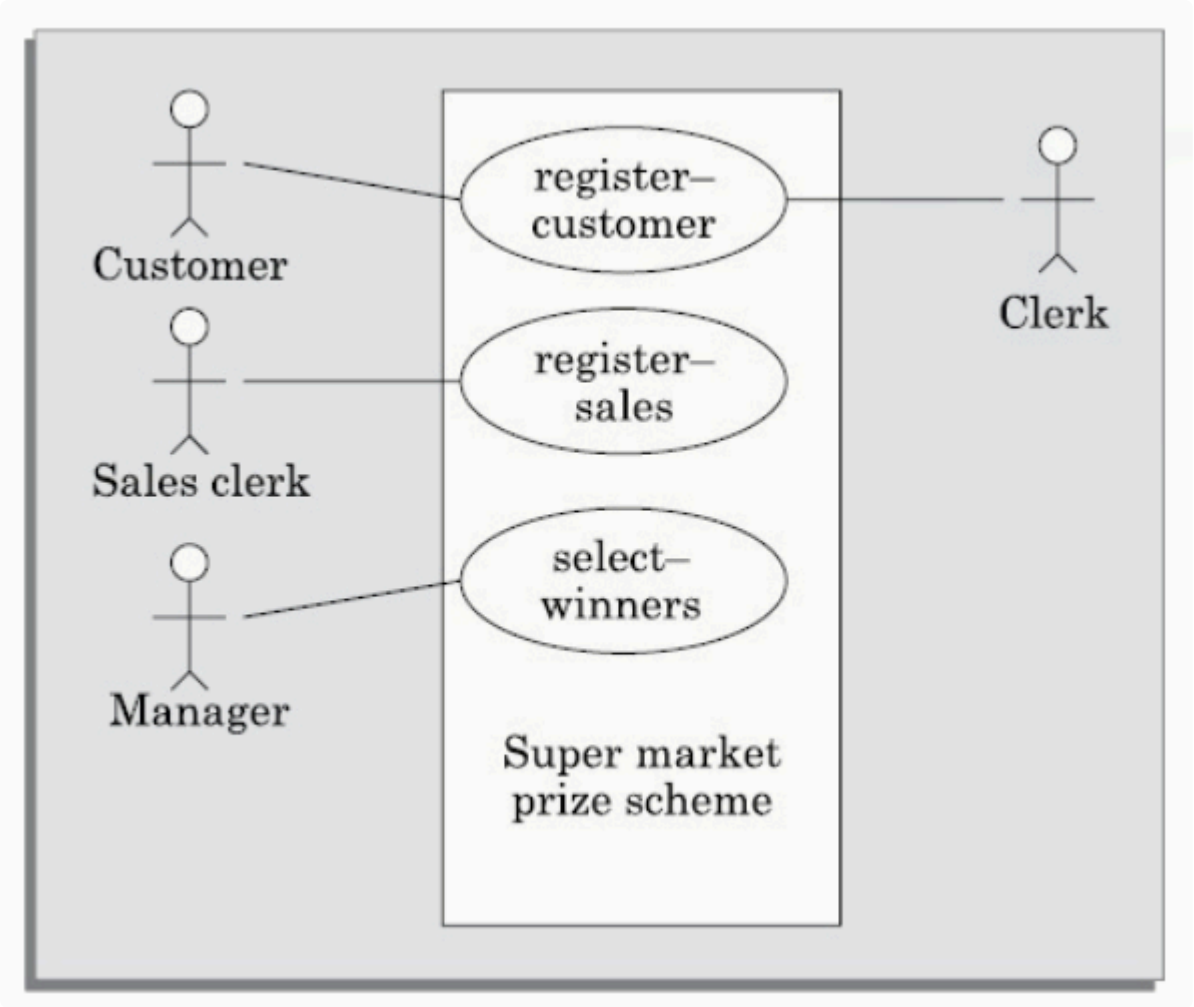
- The use case model for any system consists of a set of use cases. It represent the different ways in which a system can be used by the users.
- For example, for the library information system (LIS), the use cases could be:
  - issue-book
  - query-book
  - return-book
  - create-member
  - add-book, etc.
- The purpose of a use case is to define a piece of coherent behaviour without revealing the internal structure of the system.
- A use case consists of one main line sequence and several alternate sequences. The main line sequence represents the interactions between a user and the system that normally take place.
  For example, in the mainline sequence of the withdraw cash use

case supported by a bank ATM would be—the user inserts the ATM card, enters password, selects the amount withdraw option, enters the amount to be withdrawn, completes the transaction, and collects the amount. Typically, a variation from the mainline sequence occurs when some specific conditions hold. For example, password is invalid, the amount exceeds the account balance etc.

- The mainline sequence and each of the alternate sequences corresponding to the invocation of a use case is called a ***scenario*** of the use case.
- Normally, each use case is independent of the other use cases. However, implicit dependencies among use cases may exist because of dependencies that may exist among use cases at the implementation level due to factors such as shared resources, objects, or functions.
  For example, In the Bookshop Automation Software, update-inventory and sale-book are two independent use cases. But, during execution of sale-book there is an implicit dependency on update-inventory.

Representation of Use Cases

- A use case model can be documented by drawing a use case diagram and writing an accompanying text elaborating the drawing.
- Each use case is represented by an ellipse with the name of the use case written inside the ellipse.
- All the ellipses (i.e. use cases) of a system are enclosed within a rectangle which represents the system boundary. The name of the system being modelled (e.g., library information system) appears inside the rectangle.
- The different users of the system are represented by using stick person icons.
- Each stick person icon is referred to as an ***actor***. An actor is a role played by a user with respect to the system use.
- It is possible that the same user may play the role of multiple actors. An actor can participate in one or more use cases.
- The line connecting an actor and the use case is called the communication relationship. It indicates that an actor makes use of the functionality provided by the use case.
- Both human users and external systems can be represented by stick person icons. When a stick person icon represents an external system, it is annotated by the stereotype `<<external system>>`.



Class Diagrams

- A class diagram describes the static structure of a system. It shows how a system is structured rather than how it behaves.
- The static structure of a system comprises a number of class diagrams and their dependencies.
- The main constituents of a class diagram are classes and their relationships—generalisation, aggregation, association, and various kinds of dependencies.

Classes

- The classes represent entities with common features, i.e., attributes and operations.
- Classes are represented as solid outline rectangles with compartments.
    1. Classes have a mandatory name compartment where the name is written centred in boldface. The class name is usually written using mixed case convention and begins with an uppercase (e.g. `LibraryMember`). Object names on the other hand, are written

using a mixed case convention, but starts with a small case letter (e.g., `studentMember`).
    - Class names are usually chosen to be singular nouns.
2. Classes have optional attributes and operations compartments. Different notations are used depending on the amount of information about a class is available.

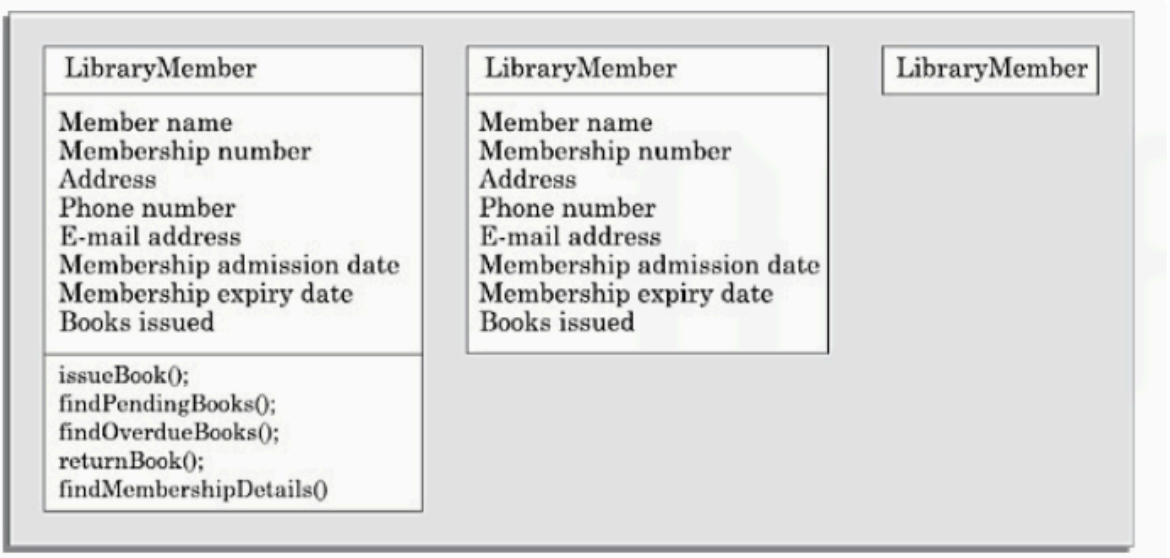


Figure 7.23: Different representations of the LibraryMember class.

Attributes

- An attribute is a named property of a class. It represents the kind of data that an object might contain.
- Attributes are listed with their names, and may optionally contain specification of their type, an initial value, and constraints.
- Attribute names are written left-justified using plain type letters, and the names should begin with a lower case letter.
- Attribute names may be followed by square brackets containing a multiplicity expression, e.g. `sensorStatus[10]` indicating the number of attributes per instance of the class.
- An attribute without square brackets must hold exactly one value. The type of an attribute is written by following the attribute name with a colon and the type name, (e.g., `sensorStatus[1]:Int`).
- The attribute name may be followed by an initialisation expression e.g. `sensorStatus[1]:Int=0`.

Operation

- The operation names are typically left justified, in plain type, and always begin with a lower case letter.

- Abstract operations are written in italics. Abstract operations are those for which the implementation is not provided during the class definition.
- The parameters of a function may have a kind specified. The kind may be “in” indicating that the parameter is passed into the operation; or “out” indicating that the parameter is only returned from the operation; or “inout” indicating that the parameter is used for passing data into the operation and getting result from the operation. The default is “in”.
- An operation may have a return type consisting of a single return type expression, e.g., issueBook(in bookName):Boolean.

Association

- Association between two classes is represented by drawing a straight line between the concerned classes.
- The name of the association is written along side the association line.
- An arrowhead may be placed on the association line to indicate the reading direction of the association.
- On each side of the association relation, the multiplicity is noted as an individual number or as a value range.
- The multiplicity indicates how many instances of one class are associated with the other.
- Value ranges of multiplicity are noted by specifying the minimum and maximum value, separated by two dots, e.g. 1..5.
- An asterisk is used as a wild card and means many (zero or more).

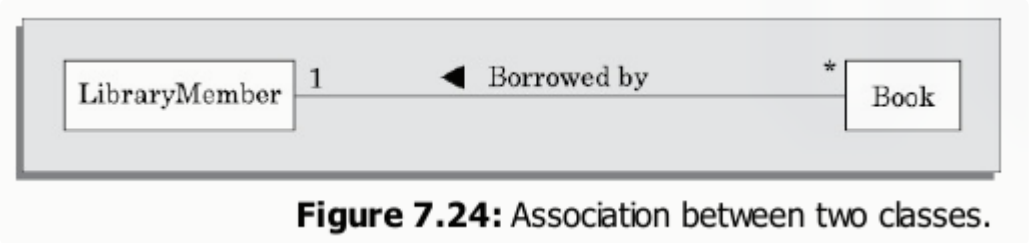


Figure 7.24: Association between two classes.

- “Many books may be borrowed by a LibraryMember”.

Aggregation

- Aggregation is a special type of association relation where the involved classes are not only associated to each other, but a whole-part relationship exists between them.

- An example of aggregation, a book register is an aggregation of book objects. Books can be added to the register and deleted as and when required.
- Aggregation is represented by an empty diamond symbol at the aggregate end of a relationship.

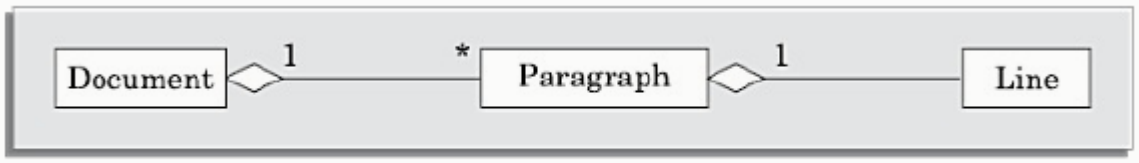


Figure 7.25: Representation of aggregation.

  - The figure represents the fact that a document can be considered as an aggregation of paragraphs. Each paragraph can in turn be considered as aggregation of lines.
  - Observe that the number 1 is annotated at the diamond end, and a * is annotated at the other end. This means that one document can have many paragraphs.
- The aggregation relationship cannot be reflexive (i.e. recursive). That is, an object cannot contain objects of the same class as itself.
- The aggregation relation is not symmetric. That is, two classes A and B cannot contain instances of each other. However, the aggregation relationship can be transitive.

Composition

- Composition is a stricter form of aggregation, in which the parts are existence-dependent on the whole.
- This means that the life of the parts cannot exist outside the whole. When the whole is created, the parts are created and when the whole is destroyed, the parts are destroyed.
- A typical example of composition is an order object where after placing the order, no item in the order cannot be changed. If any changes to any of the order items are required after the order has been placed, then the entire order has to be cancelled and a new order has to be placed with the changed items.
- The composition relationship is represented as a filled diamond drawn at the composite-end.



Figure 7.26: Representation of composition.

Inheritance

- The inheritance relationship is represented by means of an empty arrow pointing from the subclass to the superclass.
- Alternatively, when there are many subclasses of a base class, the inheritance arrow from the subclasses may be combined to a single line and is labelled with the aspect of the class that is abstracted.
- The various subclasses of a superclass can then be differentiated by means of the discriminator. The set of subclasses of a class having the same discriminator is called a ***partition***.

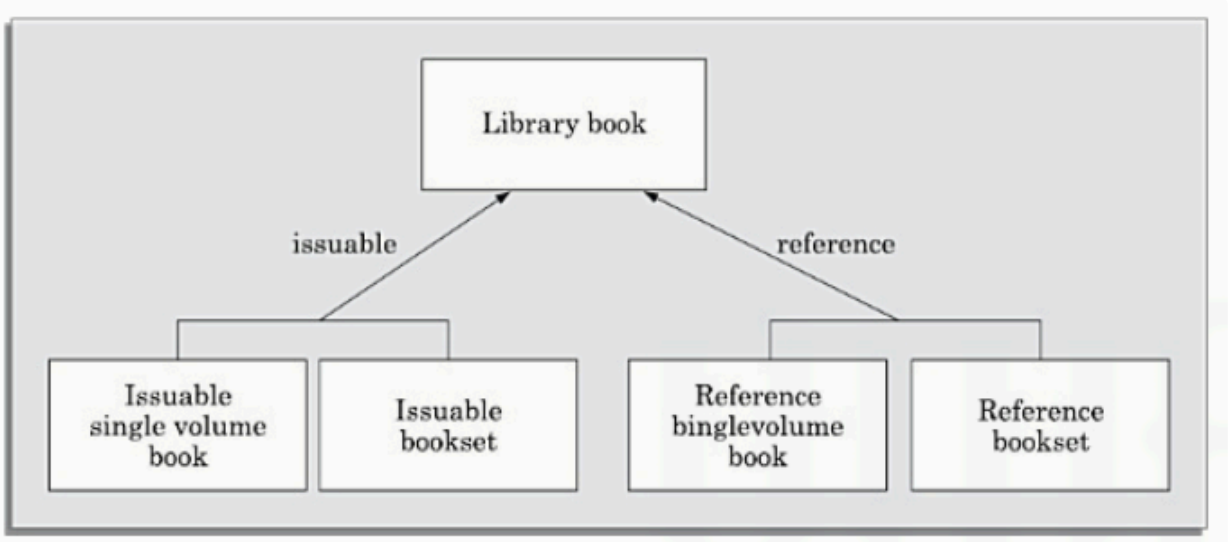


Figure 7.27: Representation of the inheritance relationship.

Dependency

- A dependency relationship is shown as a dotted arrow that is drawn from the dependent class to the independent class.

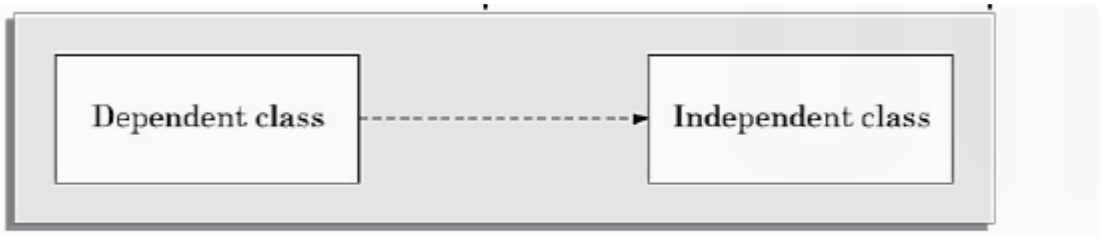


Figure 7.28: Representation of dependence between classes.

Constraints

- Constraints are typically used to describe the permissible set of values of an attribute, to specify the pre- and post-conditions for operations, to define certain ordering of items, etc.
- For example, to denote that the books in a library are sorted on ISBN number we can annotate the book class with the constraint {sorted}

Object diagrams

- Object diagrams shows the snapshot of the objects in a system at a point in time.
- Since it shows instances of classes, rather than the classes themselves, it is often called as an ***instance diagram***.
- The objects are drawn using rounded rectangles

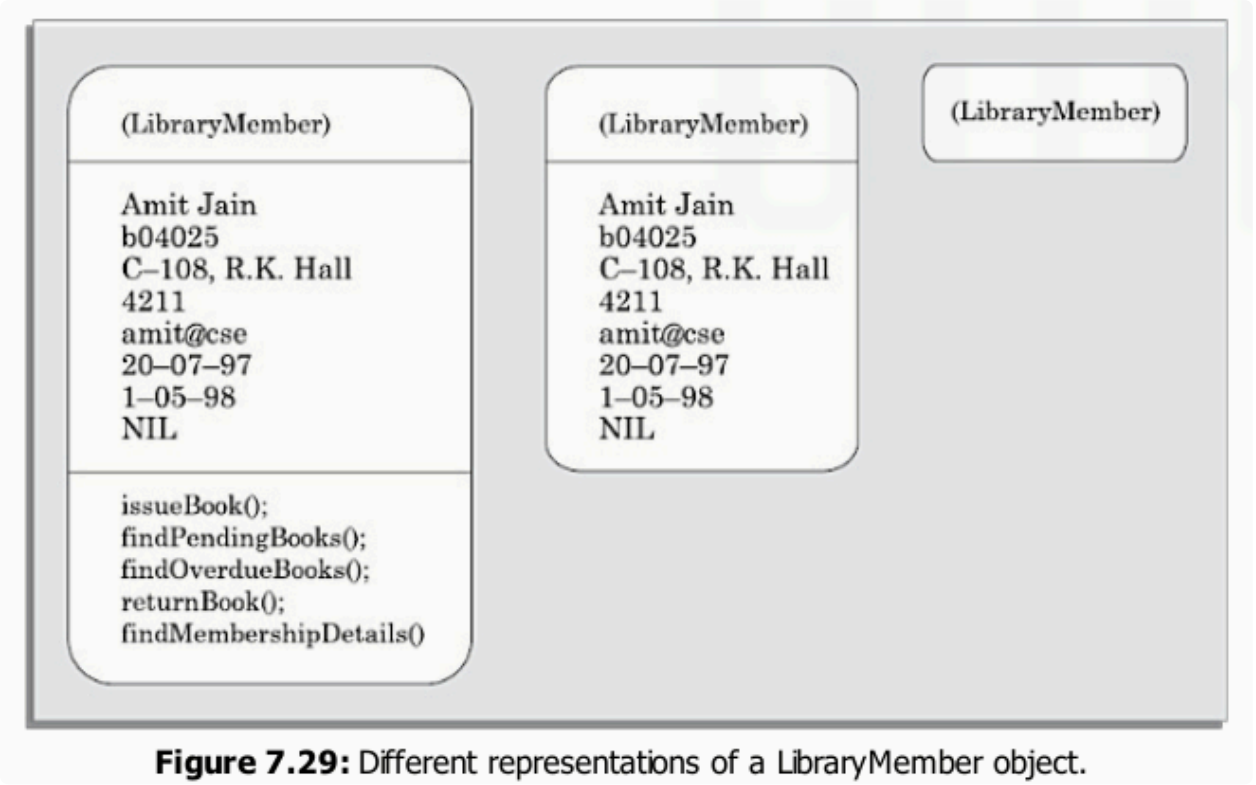


Figure 7.29: Different representations of a LibraryMember object.

Interaction Diagrams

- Interaction diagrams are models that describe how groups of objects interact among themselves through message passing to realise some behaviour or accomplish some task.
- Typically, each interaction diagram realises the behaviour of a single use case.
- For complex use cases, more than one interaction diagrams may be necessary to capture the behaviour.
- There are two kinds of interaction diagrams—**sequence diagrams** and **collaboration diagrams**.

- These two actually portray different perspectives of some behaviour of a system and different types of inferences can be drawn from them.

Sequence diagram

- A sequence diagram shows interaction among objects as a two dimensional chart.
- The chart is read from top to bottom.
- The objects participating in the interaction are shown at the top of the chart as boxes attached to a vertical dashed line.
- Inside the box the name of the object is written with a colon separating it from the name of the class and both the name of the object and the class are underlined. This signifies that we are referring any arbitrary instance of the class. For example, in Figure 7.30 :Book represents any arbitrary instance of the Book class.
- An object appearing at the top of the sequence diagram signifies that the object existed even before the time the use case execution was initiated.
- However, if some object is created during the execution of the use case then the object should be shown at the appropriate place on the

diagram where it is created.

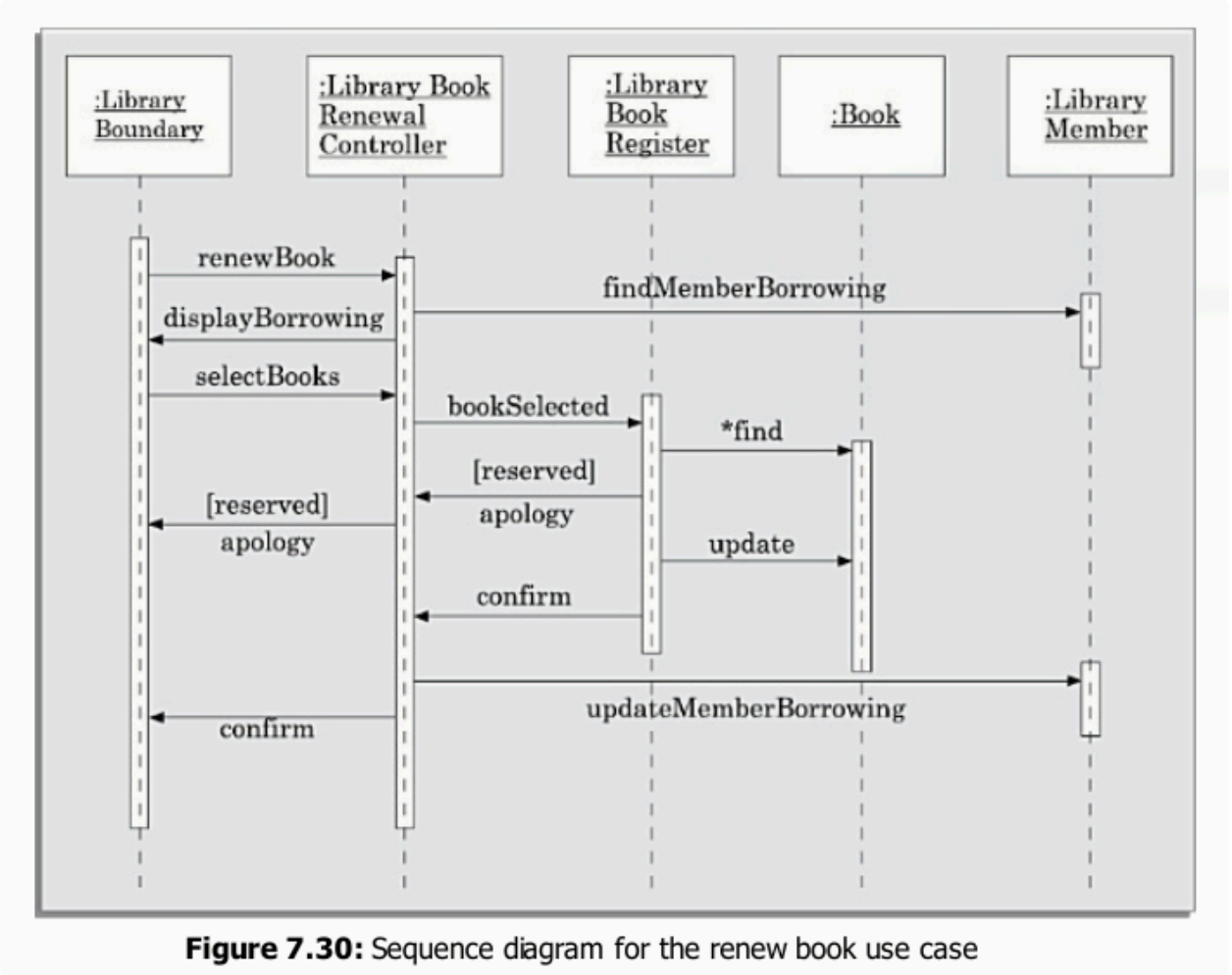


Figure 7.30: Sequence diagram for the renew book use case

- The vertical dashed line is called the object's lifeline. Any point on the lifeline implies that the object exists at that point.
- Absence of lifeline after some point indicates that the object ceases to exist after that point in time, particular point of time.
- A rectangle called the ***activation symbol*** is drawn on the lifeline of an object to indicate the points of time at which the object is active.
- Each message is indicated as an arrow between the lifelines of two objects. The messages are shown in chronological order from the top to the bottom.
- Each message is labelled with the message name.
- Some control information can also be included. Two important types of control information are:
  1. A condition (e.g., `[invalid]`) indicates that a message is sent, only if the condition is true, in this case if something is invalid.
  2. An iteration marker shows that the message is sent many times to multiple receiver objects as would happen when you are iterating over a collection or the elements of an array. You can

also indicate the basis of the iteration, e.g., `[for every book object]`.

Collaboration diagram

- A collaboration diagram shows both structural and behavioural aspects explicitly. Unlike a sequence diagram which shows only the behavioural aspects.
- The structural aspect of a collaboration diagram consists of objects and links among them indicating association.
- In this diagram, each object is also called a ***collaborator***.
- The behavioural aspect is described by the set of messages exchanged among the different collaborators.
- The link between objects is shown as a solid line and can be used to send messages between two objects.
- The message is shown as a labelled arrow placed near the link.
- Messages are prefixed with sequence numbers because they are the only way to describe the relative sequencing of the messages in this diagram.

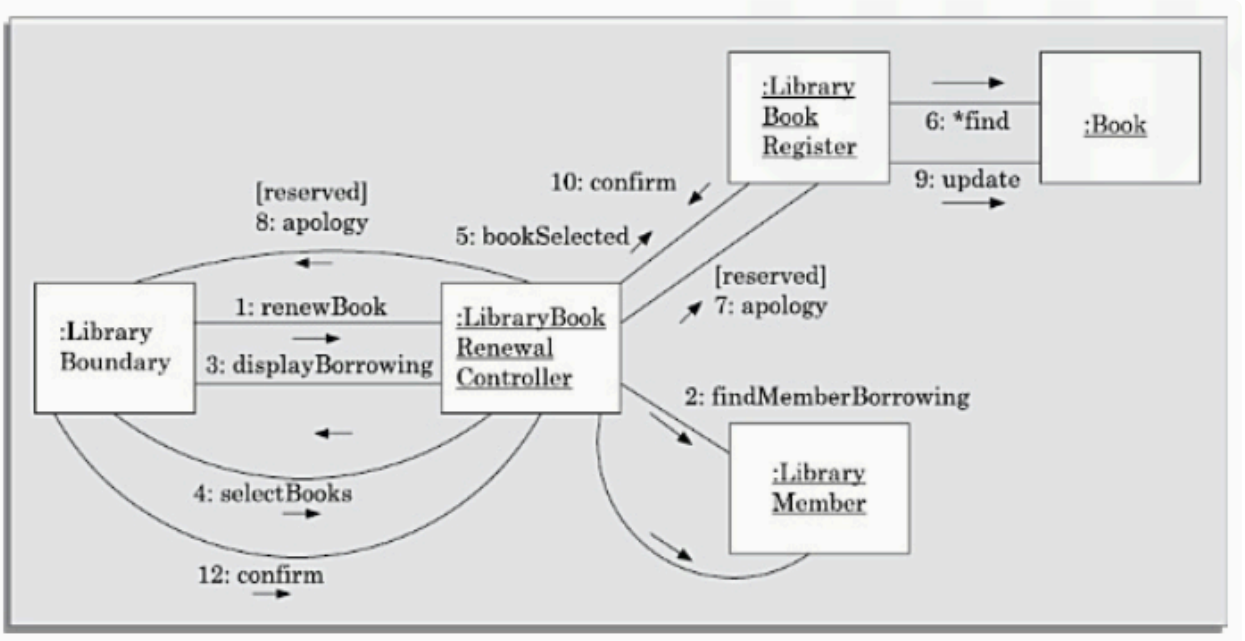


Figure 7.31: Collaboration diagram for the renew book use case.

Activity diagram

- The activity diagram focuses on representing various activities or chunks of processing and their sequence of activation.
- An activity is a state with an internal action and one or more outgoing transitions which automatically follow the termination of the internal activity.

- If an activity has more than one outgoing transitions, then exact situation under which each is executed must be identified through appropriate conditions.
- Activity diagrams are similar to the procedural flow charts. The main difference is that activity diagrams support description of parallel activities and synchronisation aspects involved in different activities.
- Parallel activities are represented on an activity diagram by using swim lanes. Swim lanes enable you to group activities based on who is performing them, e.g., academic department vs. hostel office.

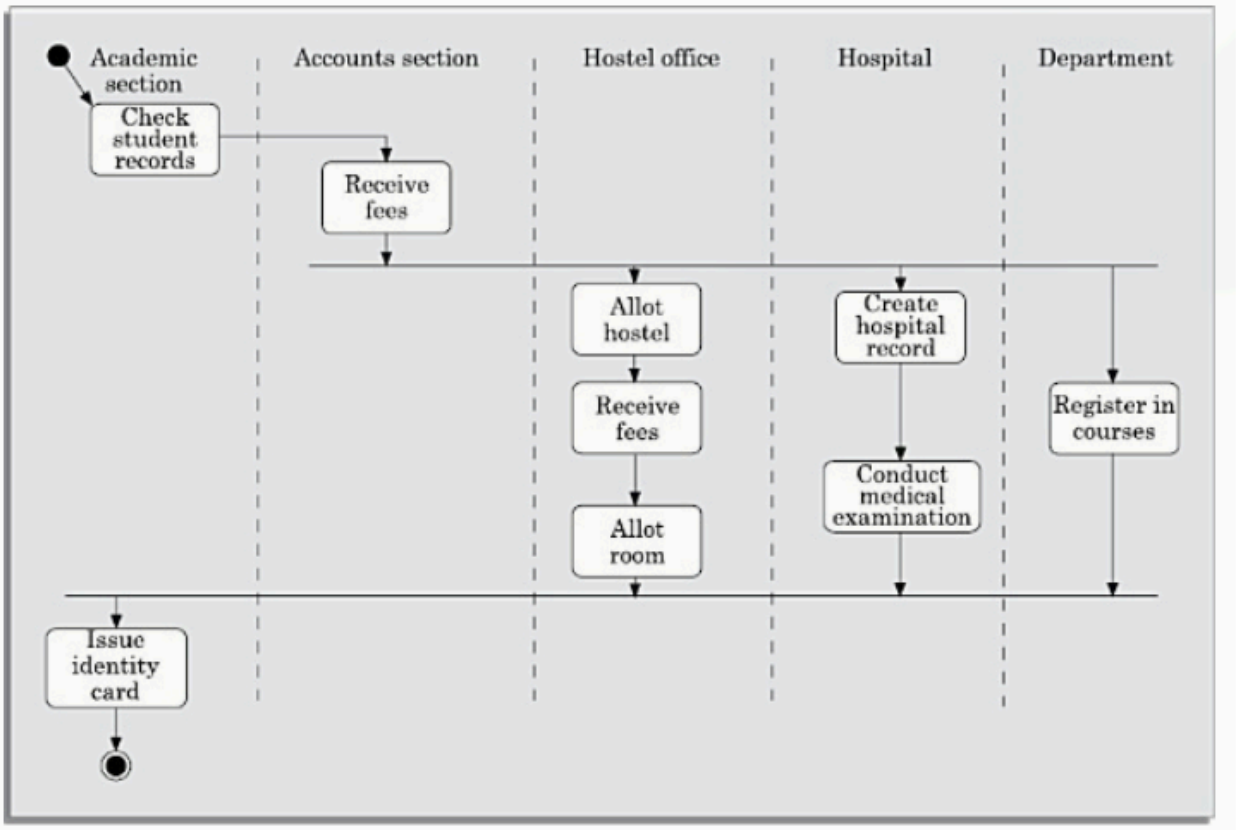


Figure 7.32: Activity diagram for student admission procedure at IIT.

  - After the fees are received at the account section, parallel activities start at the hostel office, hospital, and the Department. After all these activities complete (this is a synchronisation issue and is represented as a horizontal line), the identity card can be issued to a student by the Academic section
- Activity diagrams are normally employed in business process modelling. This is carried out during the initial stages of requirements analysis and specification.
- Activity diagrams can be very useful to understand complex processing activities involving the roles played by many components.